

Playwright Automation Testing – Step-by-Step Guide with Definitions

1. Introduction

Definition:

Playwright is an end-to-end automation testing framework developed by Microsoft. It allows testers and developers to automate web applications by controlling real browsers using code.

Explanation:

End-to-end testing means testing an application from the user's point of view. Playwright opens the browser, performs actions like clicking and typing, and verifies results just like a real user.

2. Supported Browsers

Chromium:

Definition: Chromium is an open-source browser engine used by Google Chrome and Microsoft Edge.

Why it is used: Most users access applications using Chrome or Edge, so testing on Chromium ensures wide compatibility.

Firefox:

Definition: Firefox is a browser engine maintained by Mozilla.

Why it is used: Firefox behaves differently in some scenarios, so testing ensures cross-browser stability.

WebKit:

Definition: WebKit is the browser engine behind Apple Safari.

Why it is used: Required for validating Safari and iOS browser behavior.

3. Core Architecture

Browser → Context → Page

Browser:

Definition: Represents a complete browser instance like Chrome or Firefox.

Explanation: One browser can run multiple independent sessions.

Context:

Definition: Represents an isolated browser session similar to incognito mode.

Explanation: Each context has separate cookies, cache, and storage.

Page:

Definition: Represents a single tab inside the browser.

Explanation: All user actions such as clicking, typing, and assertions happen on the page.

4. Browser Handling

Import Browsers:

```
const { chromium, firefox, webkit } = require('playwright');
```

Definition: Imports browser engines provided by Playwright.

Launch Browser:

```
const browser = await chromium.launch();
```

Definition: Starts a new browser instance.

Launch Real Chrome:

```
const browser = await chromium.launch({ channel: 'chrome' });
```

Definition: Launches the actual Google Chrome browser.

Close Browser:

```
await browser.close();
```

Definition: Shuts down the browser and frees system resources.

5. Context Management

Create Context:

```
const context = await browser.newContext();
```

Definition: Creates a new isolated session.

Explanation: Useful for testing multiple users independently.

Close Context:

```
await context.close();
```

Definition: Ends the session and clears all stored data.

6. Page Management

Create Page:

```
const page = await context.newPage();
```

Definition: Opens a new browser tab.

Handle New Tab:

```
const newPagePromise = context.waitForEvent('page');
```

```
await page.locator('a[target="_blank"]').click();
```

```
const newPage = await newPagePromise;
```

Definition: Waits for and captures a newly opened tab.

Close Page:

```
await page.close();
```

Definition: Closes the current browser tab.

7. Navigation

- Goto:

```
await page.goto('https://example.com');
```

Definition: Navigates to a specified URL.

- Reload:
await page.reload();
Definition: Refreshes the current page.
- Go Back / Forward:
await page.goBack();
await page.goForward();
Definition: Navigates browser history backward and forward.

8. Page Information

- Get URL:
page.url();
Definition: Returns the current page URL.
- Get Title:
await page.title();
Definition: Returns the title of the current page.

9. Element Locators

- locator():
page.locator('#id');
Definition: Locates an element using CSS selector.

- `getByText()`:
`page.getByText('Login');`
Definition: Locates an element by visible text.
- `getByRole()`:
`page.getByRole('button', { name: 'Login' });`
Definition: Locates elements using accessibility roles. This is the most recommended approach.

10. Waiting Mechanism

- **Explicit Wait:**
`await element.waitFor({ state: 'visible' });`
Definition: Waits until the element becomes visible.
- **Network Idle:**
`await page.waitForLoadState('networkidle');`
Definition: Waits until all network requests are completed.

11. Actions

- **Click:**
`await element.click();`
Definition: Simulates a mouse click.

- Hover:
await element.hover();
Definition: Simulates mouse hover action.
- Drag and Drop:
await page.dragAndDrop('#src', '#dest');
Definition: Drags an element from source to target.

12. Form Handling

- Fill Input:
await page.fill('#username', 'admin');
Definition: Enters text inside input fields.
- Checkbox:
await page.check('#remember');
Definition: Selects a checkbox.
- Dropdown:
await page.selectOption('#country', 'India');
Definition: Selects an option from dropdown list.

13. File Handling

- Upload File:
`await page.setInputFiles('#upload', 'file.txt');`
Definition: Uploads a file without opening OS dialog.

14. Screenshots

- Take Screenshot:
`await page.screenshot({ path: 'page.png' });`
Definition: Captures screenshot for debugging and reporting.

15. Assertions

- Import Assertion:
`import { expect } from '@playwright/test';`
Definition: Imports Playwright's assertion library.
- Common Assertions:
`await expect(page).toHaveTitle('Dashboard');`
`await expect(page).toHaveURL('https://example.com');`


```
await expect(element).toBeVisible();
```

Definition: Validates application behavior.

16. Login Automation Example

Step 1: Open login page

```
await page.goto('https://example.com/login');
```

Step 2: Enter username

```
await page.fill('#username', 'admin');
```

Step 3: Enter password

```
await page.fill('#password', '1234');
```

Step 4: Click login

```
await page.click('#login');
```

Step 5: Validate dashboard

```
await expect(page).toHaveTitle('Dashboard');
```

Explanation:

This flow simulates real user login and validates successful authentication.

17.What is Playwright Test Runner?

Definition:

Playwright Test Runner is the built-in testing framework

provided by Playwright to organize, execute, and report automated tests.

Why it is used:

- Manages test execution
- Supports parallel testing
- Generates HTML reports
- Supports retries, hooks, and fixtures
- `import { test, expect } from '@playwright/test';`

17. Playwright Test Runner

What is Playwright Test Runner?

Definition:

Playwright Test Runner is the built-in testing framework provided by Playwright to organize, execute, and report automated tests.

Why it is used:

- Manages test execution
- Supports parallel testing
- Generates HTML reports
- Supports retries, hooks, and fixtures

Test File Structure

```
import { test, expect } from '@playwright/test';
```

Definition:

Imports Playwright's test and assertion modules.

Writing a Test

```
test('Page title test', async ({ page }) => {  
  await page.goto('https://example.com');  
  await expect(page).toHaveTitle('Example Domain');  
});
```

Explanation (Step by Step):

1. test() defines a test case
2. Test name describes the validation
3. page is automatically provided
4. Navigation and assertions validate behavior

18. Headed vs Headless Mode

Headless Mode (Default)

```
const browser = await chromium.launch();
```

Definition:

Runs browser without UI.

Why used:

- Faster execution
- Used in CI/CD pipelines

Headed Mode

```
const browser = await chromium.launch({ headless: false });
```

Definition:

Runs browser with visible UI.

Why used:

- Debugging
- Learning automation visually

19. Slow Motion (Debugging)

```
const browser = await chromium.launch({ slowMo: 100 });
```

Definition:

Adds delay between actions.

Why used:

- Helps observe each step visually
- Useful for beginners and debugging

20. Test Hooks (Before & After)

What are Hooks?

Definition:

Hooks are lifecycle methods that run before or after tests.

Common Hooks

```
test.beforeEach(async ({ page }) => {  
  await page.goto('https://example.com/login');  
});
```

Definition:

Runs before every test.

Why used:

- Avoid repeating setup code
- Improves maintainability

```
test.afterEach(async ({ page }) => {  
  await page.close();  
});
```

Definition:

Runs after every test for cleanup.

21. Fixtures

What are Fixtures?

Definition:

Fixtures provide pre-configured test resources like browser, page, authentication state.

Example:

```
test('Dashboard test', async ({ page }) => {  
  await page.goto('/dashboard');  
});  
``
```

Explanation:

Playwright automatically provides:

- Browser
- Context
- Page

No manual creation required in test runner.

22. Environment Configuration

Base URL

```
use: {  
  baseURL: 'https://example.com'  
}
```

Definition:

Defines base URL for all tests.

Why used:

- Avoid repeating full URLs
- Easy environment switching

23. Screenshots on Failure

```
use: {  
  screenshot: 'only-on-failure'  
}
```

Definition:

Automatically captures screenshots when tests fail.

Why important:

- Helps identify UI issues
- Useful for reporting

24. Video Recording

use: {

video: 'retain-on-failure'

}

``

Definition:

Records test execution as video.

Why used:

- Debugging failures
- Visual proof of test behavior

25. Tracing (Powerful Feature)

use: {

trace: 'on'

}

Definition:

Records a detailed trace of test execution.

Trace Includes:

- Network logs
- DOM snapshots
- Screenshots
- Actions timeline

Why important:

Best tool for debugging flaky tests.

26. Parallel Execution

What is Parallel Testing?

Definition:

Executing multiple tests at the same time.

Why used:

- Reduces total execution time

```
test.describe.configure({ mode: 'parallel' });
```

27. Retries (Flaky Test Handling)

retries: 2

Definition:

Re-runs failed tests automatically.

Why used:

- Handles unstable environments
- Reduces false failures

28. Page Object Model (POM)

What is POM?

Definition:

A design pattern where page logic and test logic are separated.

Page Class Example

```
class LoginPage {  
  constructor(page) {  
    this.page = page;  
    this.username = page.locator('#username');  
    this.password = page.locator('#password');  
    this.loginBtn = page.locator('#login');
```

```
}
```

```
async login(user, pass) {  
  await this.username.fill(user);  
  await this.password.fill(pass);  
  await this.loginBtn.click();  
}
```

```
}
```

Why POM is important:

- Improves code reusability
- Easy maintenance
- Cleaner test files

29. API Testing with Playwright

What is API Testing?

Definition:

Testing backend APIs without UI.

```
const response = await request.get('/users');
```

Why important:

- Faster than UI testing
- Validates backend independently

30. CI/CD Integration

Where Playwright is Used

- GitHub Actions
- Azure DevOps
- Jenkins
- GitLab CI

Why important:

- Automated testing on every commit

- Ensures code quality

Final Advanced Summary (Add to Document)

- Playwright supports UI + API testing
- Built-in test runner with fixtures
- Rich debugging tools (trace, video, screenshots)
- Supports CI/CD pipelines
- Industry-ready automation framework